



8.2 · Ensembles: Random Forest y Boosting

Los métodos de ensemble combinan múltiples modelos para obtener predicciones más robustas y precisas que cualquier modelo individual. En esta unidad exploraremos los dos grandes paradigmas: **Bagging** (con Random Forest como estandarte) y **Boosting** (con Gradient Boosting y XGBoost). También aprenderemos a interpretar estos modelos mediante la importancia de variables y los valores SHAP.

UNIDAD 8.2

APRENDIZAJE AUTOMÁTICO

Índice de contenidos

01

Por qué combinar modelos

Sesgo, varianza y la sabiduría de las masas

02

Bagging y Random Forest

Bootstrap aggregating y decorrelación de árboles

03

Importancia de variables

MDI, permutación y comparativa

04

Boosting y Gradient Boosting

Aprendizaje secuencial y descenso por gradiente

05

XGBoost, Early Stopping y SHAP

Parámetros clave, regularización e interpretabilidad

¿Por qué combinar modelos?

Un único modelo predictivo siempre arrastra limitaciones: alto sesgo si es demasiado simple, o alta varianza si es demasiado complejo. La idea central de los **métodos de ensemble** es combinar múltiples modelos «débiles» para construir uno «fuerte», reduciendo simultáneamente ambos problemas.



El dilema sesgo–varianza y los ensembles

Problema del modelo único

- **Alto sesgo:** subajuste, no captura la señal
- **Alta varianza:** sobreajuste, memoriza el ruido
- Difícil equilibrar ambos con un solo estimador

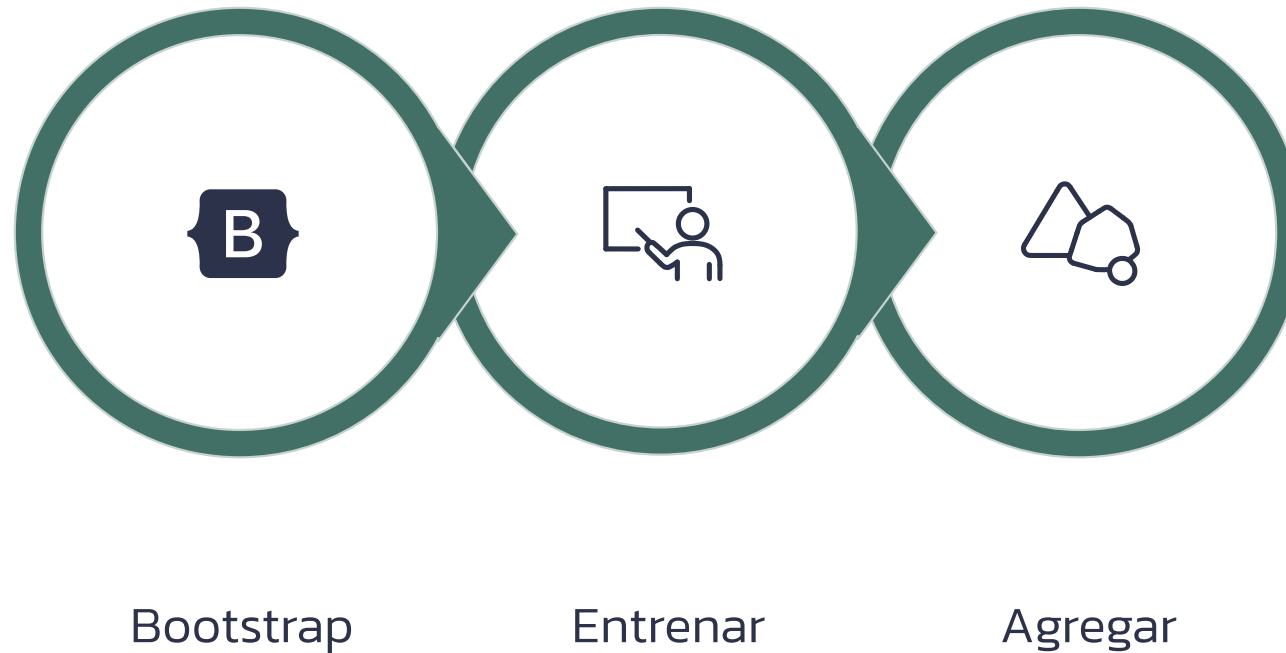
Solución: combinar modelos

- **Promediado:** reduce la varianza sin aumentar el sesgo
- **Boosting:** reduce el sesgo iterativamente
- **Diversidad:** los errores de cada modelo son independientes

❏ La clave del éxito está en que los modelos base cometan **errores distintos** entre sí.

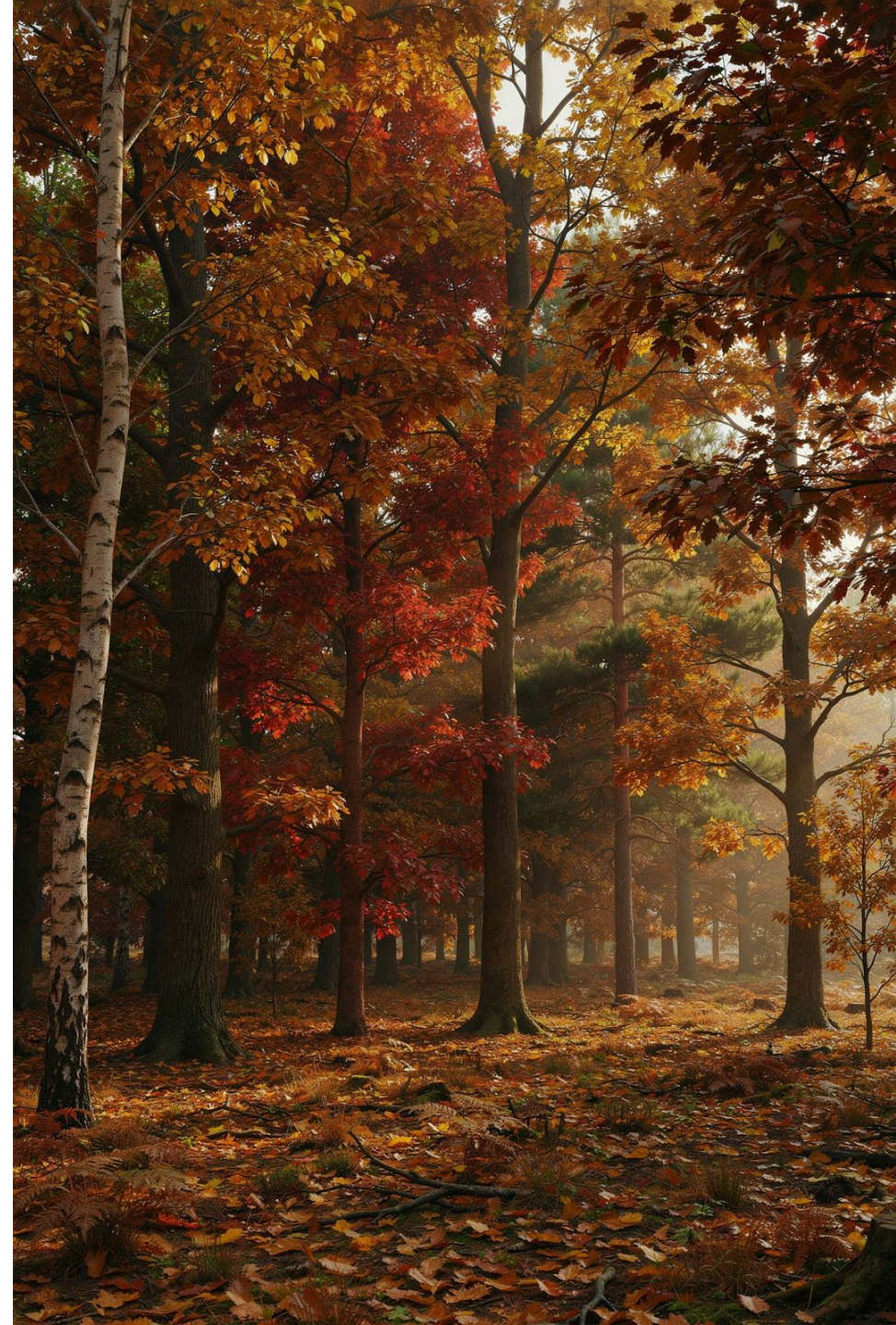
Bagging: Bootstrap Aggregating

El **Bagging** (Bootstrap Aggregating) entrena múltiples modelos base de forma independiente y en paralelo, cada uno sobre una muestra aleatoria con reemplazamiento del conjunto de entrenamiento. Las predicciones finales se obtienen por votación mayoritaria (clasificación) o promedio (regresión).



Random Forest

Random Forest extiende el Bagging añadiendo una segunda fuente de aleatoriedad: en cada división de cada árbol, solo se considera un subconjunto aleatorio de **m** características (típicamente $\sqrt{p}$ para clasificación y $p/3$ para regresión). Esto **decorrelaciona** los árboles, haciendo que sus errores sean más independientes y que el promedio sea más estable.



Random Forest: parámetros fundamentales

n_estimators

Número de árboles. Más árboles → mayor estabilidad. El error converge; no hay sobreajuste por aumentar este valor, solo coste computacional.

max_features

Variables candidatas por split. Controla la decorrelación. Valores bajos → mayor diversidad pero mayor sesgo individual.

max_depth

Profundidad máxima de cada árbol. Los árboles completos (sin límite) funcionan bien con Bagging gracias al promediado.

oob_score

Estimación gratuita del error de generalización usando las muestras *out-of-bag*. Equivale a una validación cruzada sin coste extra.

Importancia de variables en Random Forest

Random Forest ofrece dos medidas estándar para cuantificar la relevancia de cada predictor:

MDI — Mean Decrease in Impurity

Suma la reducción de impureza (Gini o varianza) provocada por cada variable a lo largo de todos los árboles y todos los nodos. Es rápida de calcular pero puede **favorecer variables de alta cardinalidad** y las codificadas como continuas.

Importancia por permutación

Aleatoriza los valores de una variable en el conjunto de validación y mide cuánto aumenta el error. Es más costosa pero **más fiable y agnóstica al modelo**. Disponible en scikit-learn como `permutation_importance`.

- ❗ Cuando las variables tienen escalas o cardinalidades muy distintas, prefiere la importancia por permutación para conclusiones más robustas.



Boosting: aprendizaje secuencial

A diferencia del Bagging, el **Boosting** entrena los modelos base de forma **secuencial**: cada nuevo modelo se centra en los ejemplos que los anteriores clasificaron peor. El resultado final es una suma ponderada de modelos débiles que colectivamente forman un estimador potente con bajo sesgo.

Gradient Boosting: el algoritmo

Gradient Boosting enmarca el Boosting como un problema de **optimización funcional**: en cada iteración se ajusta un árbol a los **residuos negativos del gradiente** de la función de pérdida, avanzando en la dirección que más reduce el error.



Bagging vs. Boosting: comparativa directa

Bagging



1. Orden de Entrenamiento
En paralelo
independientes



2. Objetivo Principal
Reducir varianza



3. Modelos Base
Árboles profundos



4. Riesgo de Sobreajuste
Rara vez sobreajusta



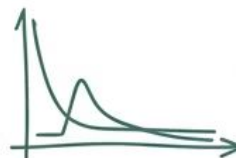
5. Velocidad
Rápido
paralelizable

6. Ejemplo
Random Forest

Boosting



Secuencial
dependientes



2. Objetivo Error
Reducir sesgo



3. Modelo Base
Tocones (shallow)



Puede sobreajustar
Puede sobreajustar (muchas iteraciones)



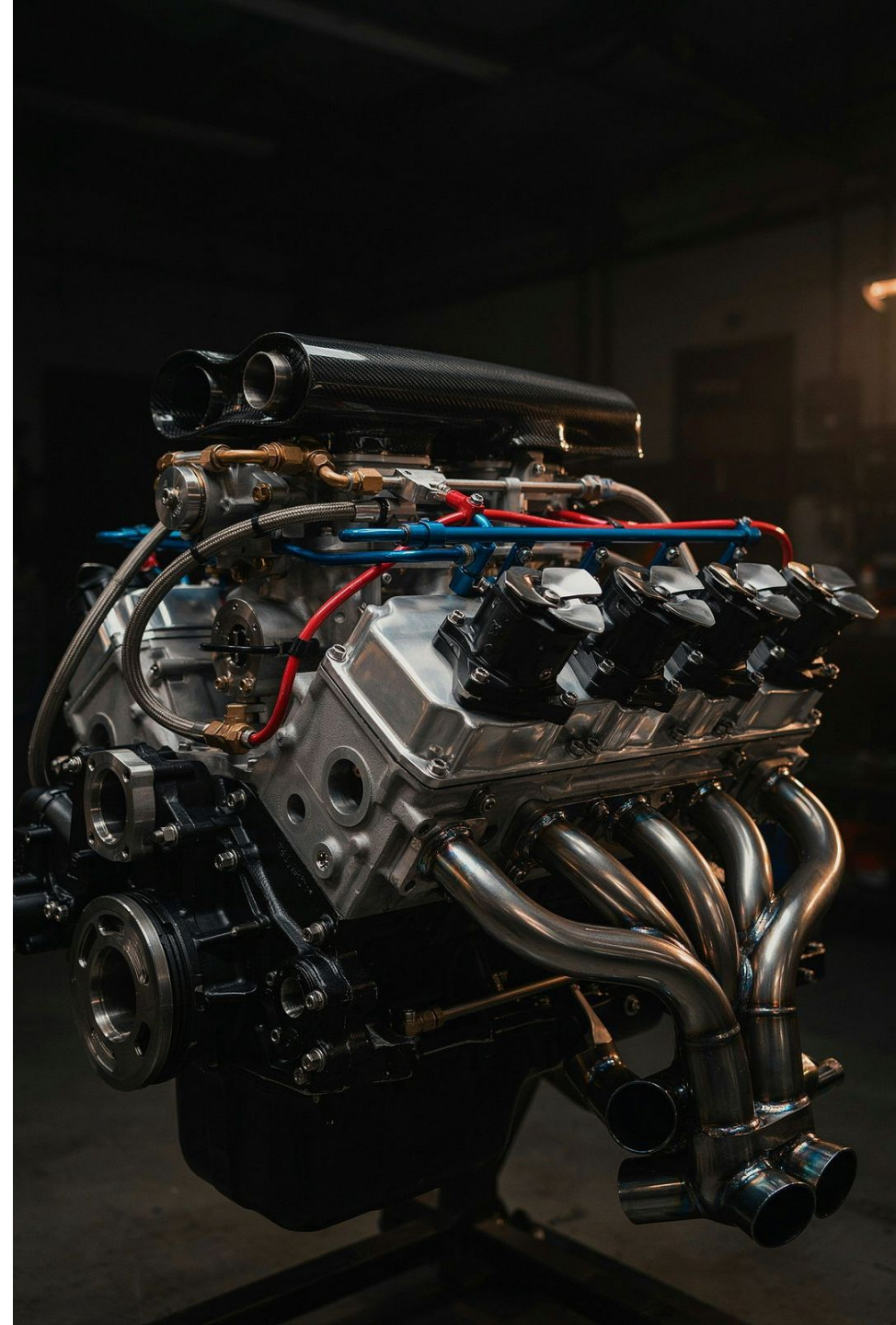
5. Velocidad
Más lento
secuencial



6. Ejemplo
XGBoost

XGBoost: Extreme Gradient Boosting

XGBoost es la implementación de Gradient Boosting más utilizada en competiciones y producción. Incorpora **regularización L1 y L2** en la función objetivo, lo que penaliza modelos complejos y reduce el sobreajuste. Añade además un término de complejidad basado en el número de hojas y sus pesos. Su implementación en C++ con soporte nativo para paralelismo lo hace significativamente más rápido que las implementaciones clásicas.



Parámetros clave de XGBoost

`learning_rate` (η)

Escala la contribución de cada árbol. Valores pequeños (0.01–0.1) combinados con muchos árboles suelen generalizar mejor.

`n_estimators`

Número de rondas de boosting. Con *early stopping* no hace falta ajustarlo manualmente.

`max_depth`

Profundidad máxima de cada árbol. Valores entre 3 y 6 son habituales; árboles más profundos aumentan el riesgo de sobreajuste.

`subsample` y `colsample`

Fracción de filas y columnas muestreadas por árbol. Introducen aleatoriedad al estilo Random Forest, actuando como regularización.

`lambda` y `alpha`

Regularización L2 y L1 sobre los pesos de las hojas. Penalizan modelos complejos directamente en la función objetivo.

Early Stopping: detener en el momento justo

¿Qué es?

El **Early Stopping** monitoriza el error en un conjunto de validación tras cada ronda de boosting. Si el error no mejora durante `early_stopping_rounds` iteraciones consecutivas, el entrenamiento se detiene automáticamente y se recupera el modelo de la mejor ronda.

Ventajas

- Evita el sobreajuste sin necesidad de ajustar `n_estimators`
- Reduce el tiempo de entrenamiento en modelos grandes
- Funciona como validación cruzada ligera integrada

Uso en XGBoost

Se activa pasando un `eval_set` al método `fit()` y especificando `early_stopping_rounds`. El modelo entrenado expone `best_iteration` para conocer la ronda óptima.

- ❏ Usa siempre un conjunto de validación **separado** del test para no sesgar la evaluación final.

¿Cuándo usar Random Forest y cuándo XGBoost?



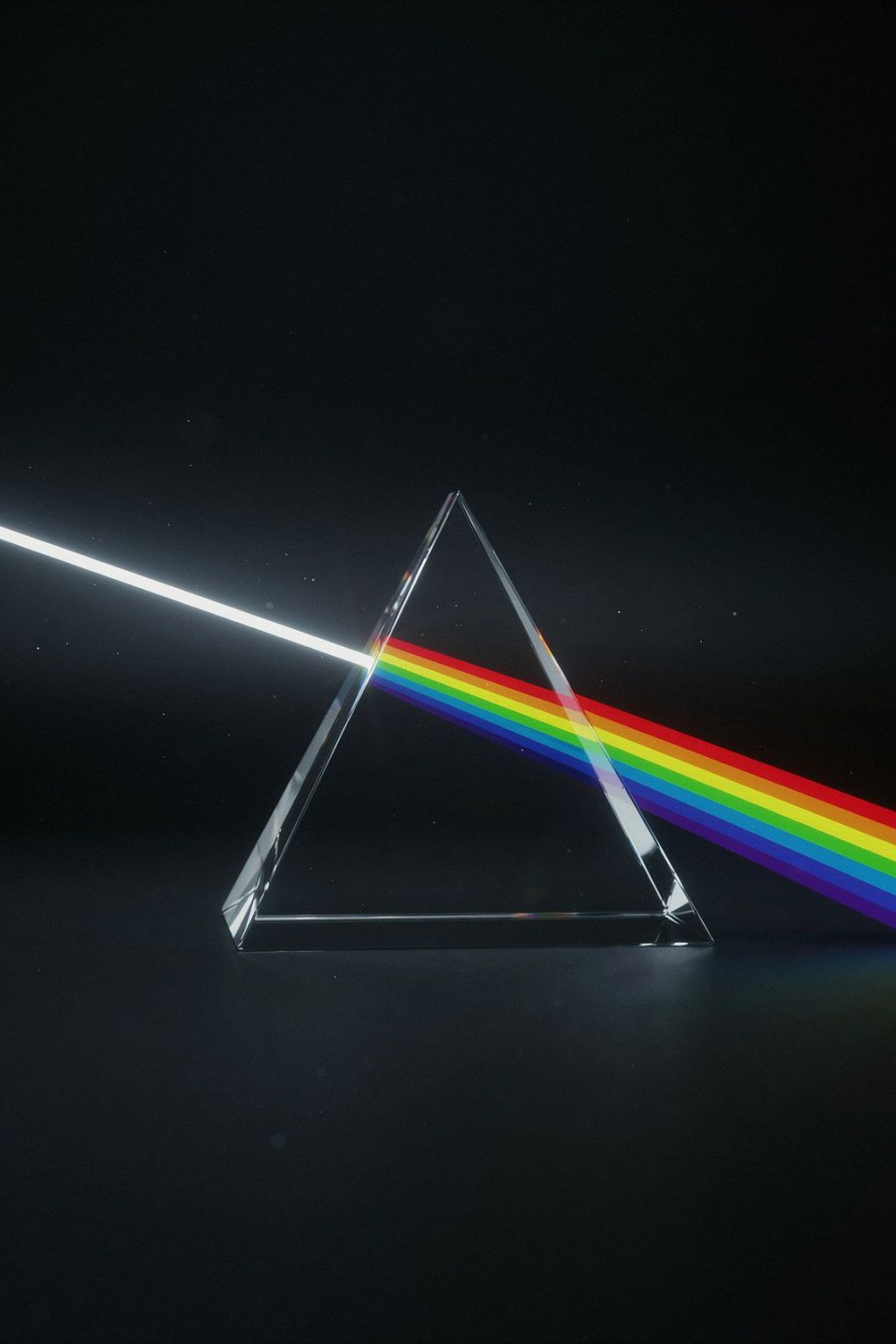
Elige Random Forest cuando...

- Necesitas un modelo robusto con mínimo ajuste de hiperparámetros
- El tiempo de entrenamiento es limitado (paralelizable)
- El conjunto de datos tiene muchas variables irrelevantes
- Priorizas estabilidad y reproducibilidad



Elige XGBoost cuando...

- Buscas el máximo rendimiento predictivo
- Dispones de tiempo para ajustar hiperparámetros
- El problema es tabular y el error de sesgo es dominante
- Participas en competiciones de machine learning



Introducción a SHAP

SHAP (SHapley Additive exPlanations) aplica la teoría de juegos cooperativos — concretamente los valores de Shapley— para atribuir a cada variable su contribución exacta sobre la predicción de una observación individual. A diferencia de las importancias globales, SHAP explica **por qué** el modelo predijo un valor concreto para un ejemplo concreto.

Valores SHAP: interpretación y visualizaciones

Propiedades formales

- **Eficiencia:** los valores suman exactamente la diferencia entre la predicción y la media global
- **Simetría:** variables con el mismo impacto reciben el mismo valor
- **Linealidad:** los valores son aditivos entre modelos

Tipos de gráficos SHAP

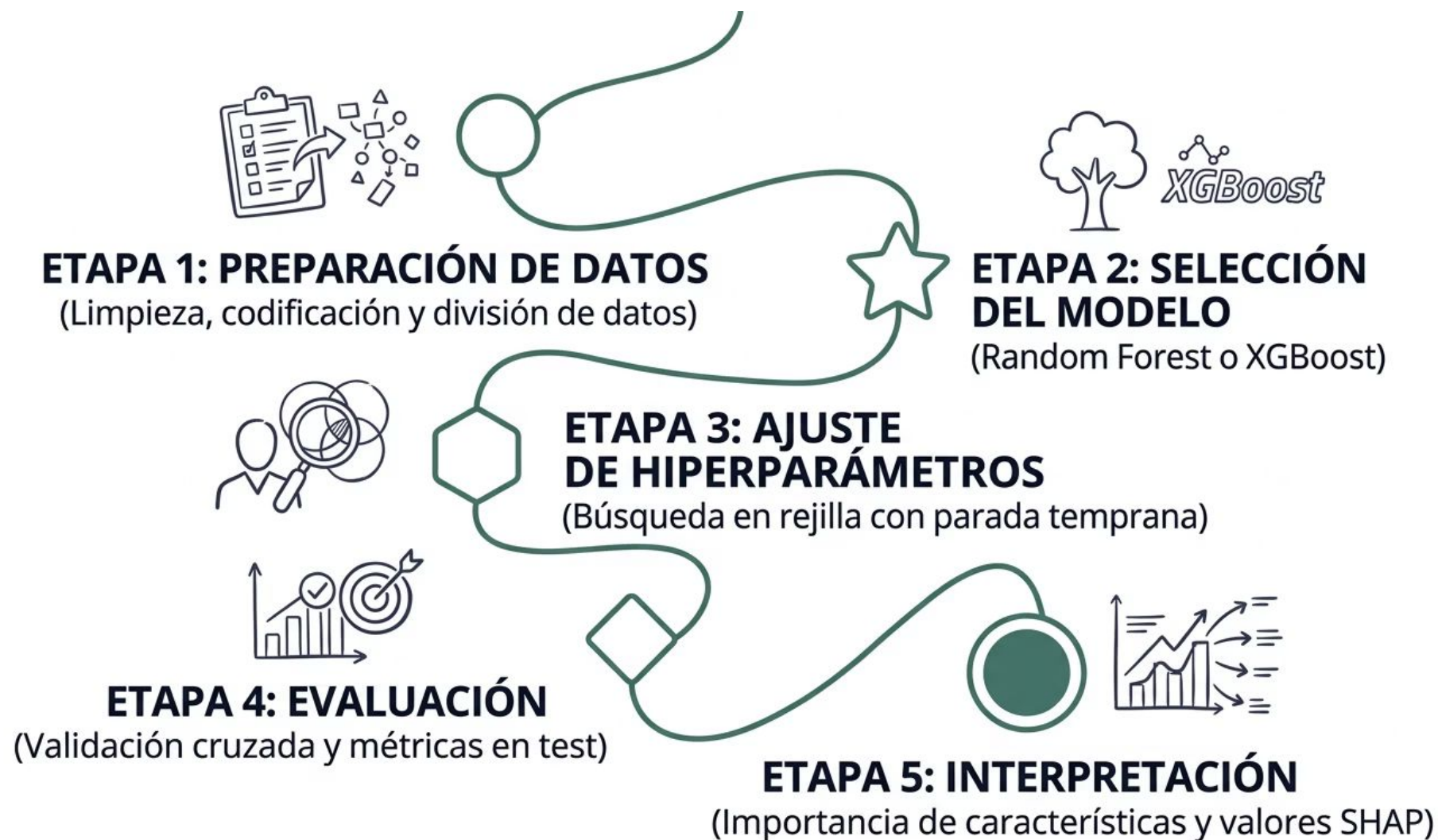
- **Waterfall plot:** desglose para una predicción individual
- **Beeswarm plot:** distribución global de impactos
- **Dependence plot:** relación entre una variable y su SHAP
- **Force plot:** visualización compacta de contribuciones

Compatibilidad con ensembles

La librería `shap` incluye el **TreeExplainer**, un algoritmo optimizado para árboles de decisión que calcula los valores exactos en tiempo polinómico. Es compatible con Random Forest, XGBoost, LightGBM y CatBoost.

📄 SHAP permite pasar de «el modelo es una caja negra» a «entiendo exactamente por qué predijo esto».

Pipeline de ensemble: de los datos a la predicción



Errores frecuentes y buenas prácticas

No usar early stopping en Boosting

Entrenar XGBoost con un número fijo de iteraciones sin monitorizar validación lleva inevitablemente al sobreajuste en cuanto `n_estimators` es elevado.

Confiar solo en MDI para importancia

La importancia MDI puede estar sesgada hacia variables continuas o de alta cardinalidad. Complementa siempre con importancia por permutación o SHAP.

Escalar `learning_rate` con `n_estimators`

Reducir a la mitad el `learning_rate` y duplicar `n_estimators` suele mantener la calidad y mejorar la generalización.

Validar con OOB score en Random Forest

El error *out-of-bag* es un estimador gratuito de la generalización. Actívalo con `oob_score=True` para ahorrar tiempo frente a la validación cruzada completa.

Resumen y puntos clave



Bagging reduce varianza

Random Forest paralela árboles diversos mediante bootstrap y selección aleatoria de variables.



Boosting reduce sesgo

Gradient Boosting y XGBoost aprenden secuencialmente de los errores residuales de cada iteración.



Importancia de variables

Usa MDI como primera exploración y permutación o SHAP para conclusiones robustas e interpretables.



Early stopping

Detiene el entrenamiento cuando el error de validación deja de mejorar, evitando sobreajuste automáticamente.



SHAP para explicabilidad

Los valores Shapley desglosan cada predicción individual, haciendo los ensembles transparentes y auditables.

- ❏ Los ensembles son hoy el método más utilizado en problemas tabulares del mundo real. Dominarlos es una habilidad esencial para cualquier data scientist.